

SHRI KRISHNASWAMY COLLEGE FOR WOMEN

TN SKILL PROJECT [SEM 3]

**EduGenie:Google Gemini Powered Learning
Assistant**

TEAM ID :SWTID-2026-4092

TEAM LEADER :LIKKITHA S

TEAM MEMBER :Geshna S

TEAM MEMBER :Hajira H

TEAM MEMBER :Harini B

TEAM MEMBER :Keerthana K

Description

The ContentGen backend is a powerful, secure, and intelligent REST API platform designed to simplify blog creation, content management, and AI-assisted writing. Built with Node.js and Express.js, the application uses MongoDB as its primary database with Mongoose ODM for efficient data modeling, validation, and seamless database interactions.

The platform integrates Google Gemini AI to provide intelligent features such as AI-powered blog content generation, AI FAQ assistance, AI WeatherWise insights, and AI FitTrack recommendations, enabling users to create high-quality, context-aware content with minimal effort.

Security is a core aspect of the architecture, implementing JWT (JSON Web Tokens) for stateless authentication, role-based access control (Admin, Editor, Author, Reader) for authorization, and bcrypt.js for secure password hashing. The backend also supports comprehensive blog management, categories and tags, comment management, media metadata handling, advanced search capabilities, and centralized error handling. Additional security mechanisms such as request validation, input sanitization, rate limiting, and structured logging ensure the platform remains reliable, scalable, and resilient against common web vulnerabilities.

Designed with a modular and scalable architecture, ContentGen serves as a modern AI-powered Content Management System (CMS) that combines traditional blogging capabilities with Generative AI to deliver a smarter, faster, and more personalized content creation experience.

Scenario-Based Case Study

Background

Priya is an active content creator managing multiple blogs who encounters difficulty organizing posts, scheduling publications, maintaining categories/tags, and effectively monitoring reader engagement.

Problem

- Difficulty coordinating multiple post drafts, categories, and tags.
- Limited insight into post performance metrics and reader engagement analytics.
- Slow, manual workflows for content approval and comment moderation.
- Absence of a standardized system for managing media uploads and associated metadata.
- Lack of collaborative, role-based access levels for editors, authors, and administrators.

Solution

The BlogNest backend offers a centralized, secure API wrapper that separates internal application logic from frontend delivery systems. It incorporates an internal permission matrix to manage dedicated services for publishing schedules, category taxonomy, media path registration, and fine-grained user roles.

Usage

- **Author Usage:** Drafts blog posts, registers media file paths, organizes content ideas, and monitors personal post-performance data.
- **Editor/Admin Usage:** Accesses guarded endpoints to review approval queues, update publishing states, moderate user comments, and manage user role upgrades.
- **Reader Usage:** Queries public routing pathways to read published articles and uses comment endpoints to engage with posts.

Outcome

Delegating administrative workflows to a structured backend reduces publishing delays. Schema validation enforces strict data integrity, security middleware prevents unauthorized privilege changes, and persistent tracking delivers raw analytics data to accurately evaluate post performance.

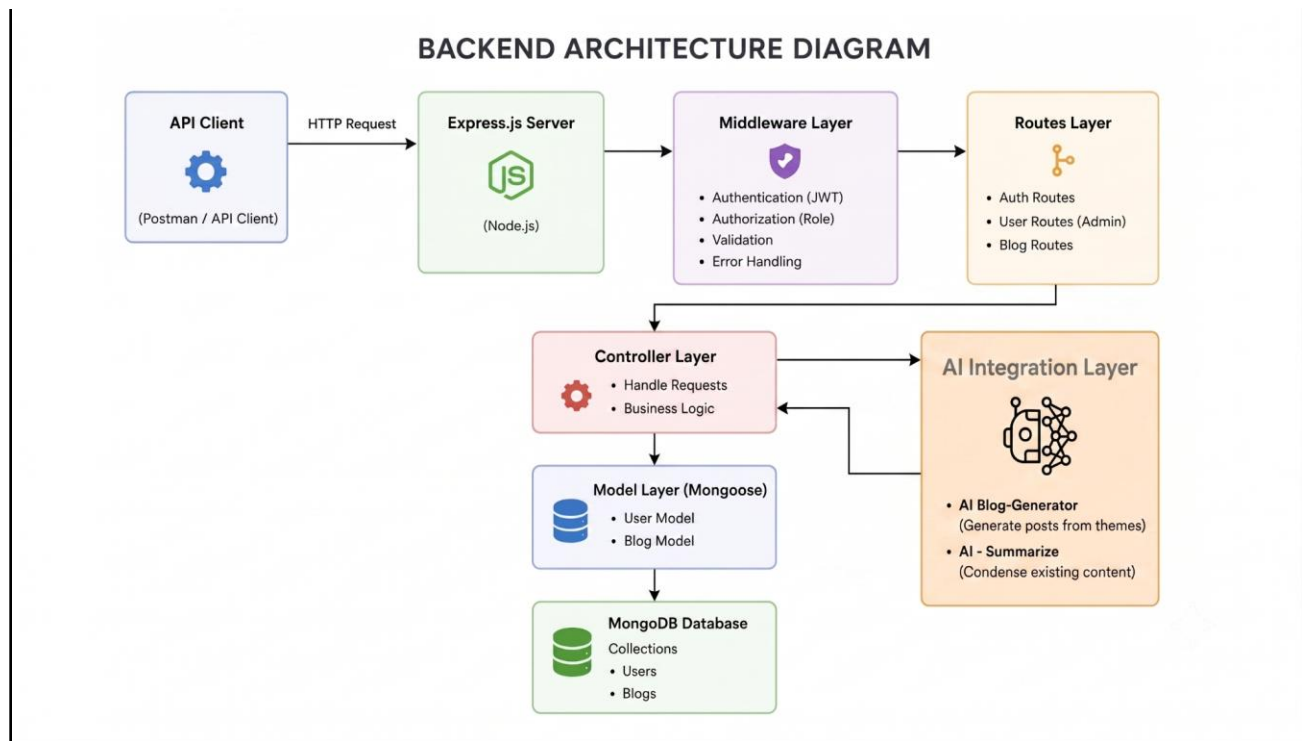
1. Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux (fully supports cross-platform operations).
- **Node.js (v16 or above):** JavaScript runtime execution engine powering server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to handle project operational dependencies.
- **Express.js:** Lightweight web routing framework used to build backend RESTful endpoints.
- **MongoDB:** Document-oriented NoSQL database storing users, posts, categories, comments, and analytics data.
- **Postman:** API verification tool used to test and validate schema responses across protected route middleware.
- **Code Editor:** Visual Studio Code or similar modern IDE environment.

2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended to smoothly run concurrent instances of MongoDB, background processes, and application nodes).
- **Storage:** 1 GB of available disk workspace.

Epic:-1 PROJECT ARCHITECTURE



The BlogNest application exposes structured API endpoints handled by a layered server architecture. Inbound client requests pass through security and validation filters before interacting with internal business services or data stores.

Components Definition

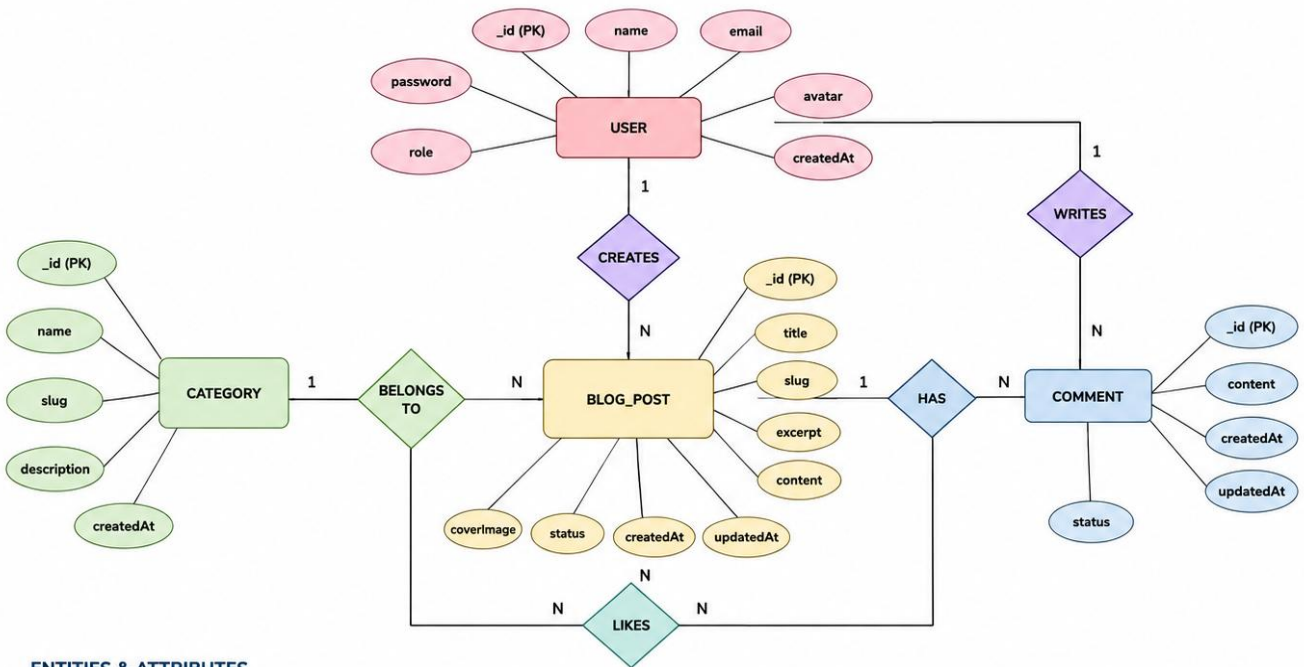
- **Interface:** API platforms like Postman send queries to designated route paths to request or send JSON formatted entities.
- **Express Server Gateway:** Listens on active network ports, maps incoming HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads, and manages CORS configuration headers.

- **Authentication & Role Matrix Middleware:** Extracts bearer tokens from request headers, verifies user signature contexts, and confirms whether the assigned role meets authorization requirements for restricted route paths.
- **Granular Services (Logic Modules):** Enforces isolated domain rules for blog post lifecycles (Draft, Pending Approval, Scheduled, Published), comment moderation queues, and operational analytics reporting.
- **Database Interface Layer:** Mongoose translates backend application commands into optimized native MongoDB queries.
- **AI Flow (Service Integration):** Powers automated content workflows, including smart blog draft generation and real-time text summarization. This module connects to external AI models through dedicated endpoints
- **BlogGenerator with AI:** A specialized service that automates the generation of blog content, which integrates with the AI Flow service to streamline the drafting process.

Entity-Relationship (ER) Diagram Description

The underlying document structure manages several collections bound by logical application references:

BlogNest – ER Diagram



ENTITIES & ATTRIBUTES

USER

- _id (PK)
- name
- email
- password
- avatar
- role (admin/user)
- createdAt

BLOG_POST

- _id (PK)
- title
- slug
- excerpt
- content
- coverImage
- status (draft/published)
- createdAt
- updatedAt

CATEGORY

- _id (PK)
- name
- slug
- description
- createdAt

COMMENT

- _id (PK)
- content
- status (active/inactive)
- createdAt
- updatedAt

RELATIONSHIPS

- USER 1 —> CREATES —> N BLOG_POST
(A user can create many blog posts)
- CATEGORY 1 —> BELONGS TO —> N BLOG_POST
(A blog post belongs to one category)
- BLOG_POST 1 —> HAS —> N COMMENT
(A blog post can have many comments)
- USER 1 —> WRITES —> N COMMENT
(A user can write many comments)
- USER N —> LIKES —> N BLOG_POST
(Many users can like many blog posts)

1. User Entity

- _id (Primary Key - ObjectId)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)
- role (String, Default: "reader")

2. Post Entity

- _id (Primary Key - ObjectId)
- userID (Foreign Key referencing User Collection)

- **name** (String, Required - represents Title)
- **photo** (Object/String pathway link)
- **message** (String - Core content body)
- **likes** (Number, Default: 0)
- **timestamps** (True)

Key Structural Mappings

- **User to Post (One-to-Many):** An author can submit multiple blog documents.
- **User to Comment (One-to-Many):** Readers/Users can append multiple response comments across independent post nodes.
- **Post to Comment (One-to-Many):** A single blog document stores individual referenced comment submissions.

Key Features

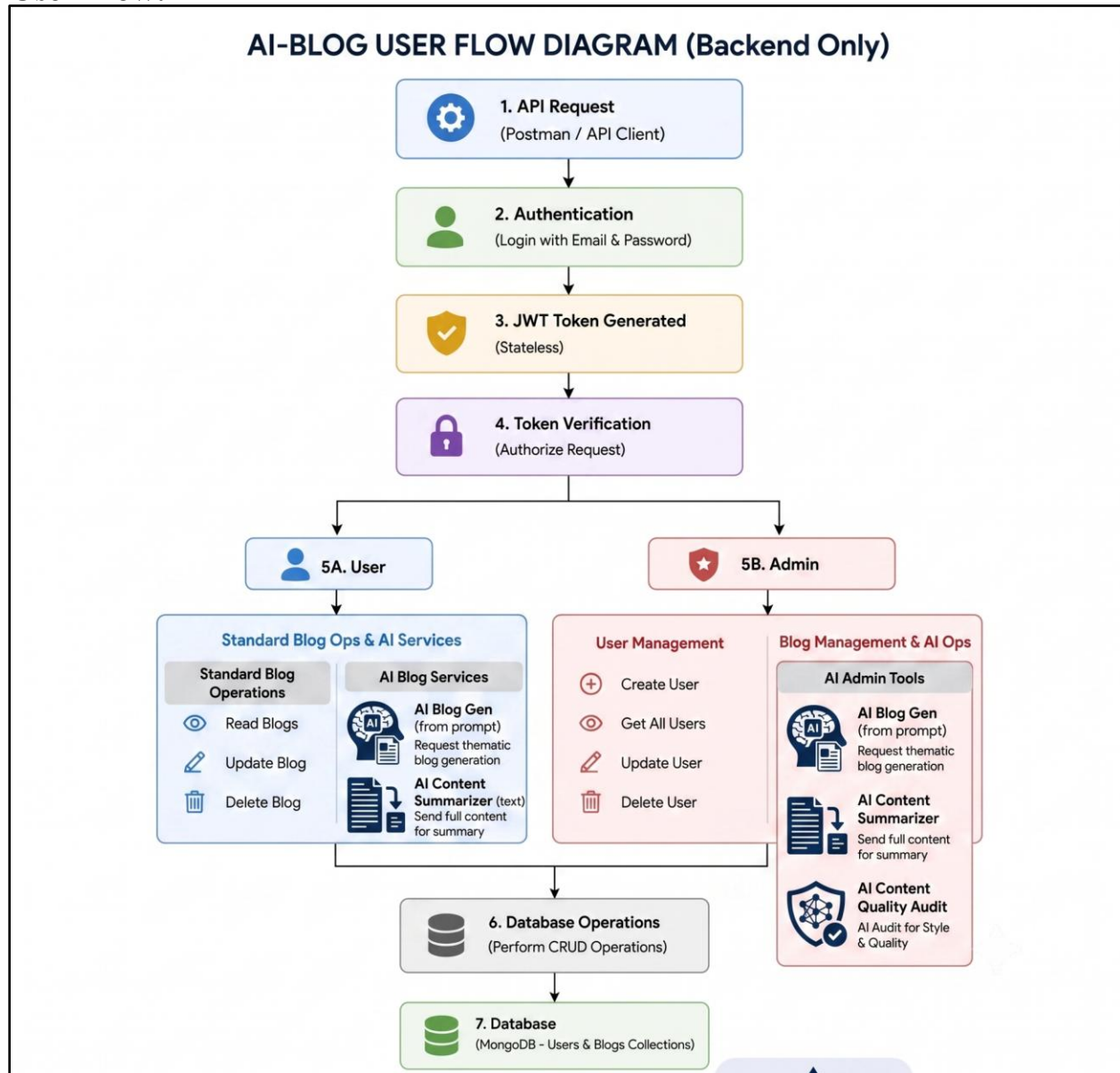
- **Role-Based Route Guarding:** Strict token processing blocks unauthorized tiers from modifying core settings.
- **CRUD Blog Execution Engine:** Complete schema management over content definitions, status states, and asset assignment paths.
- **Categorization System:** Native tag indices facilitate filtering and querying for enhanced discoverability.
- **Comment Moderation Logic:** Protected entry points for updating moderation flags (approved, pending, spam).
- **Sanitization Safeguards:** Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections.
- **I-Driven Blog Generation Pipeline:** Integrates external Large Language Model (LLM) abstractions and context-aware prompt templates to synthesize full-length, structured blog posts from high-level thematic inputs.
- **Intelligent Content Summarization Engine:** Leverages semantic compression layers to parse extensive article payloads, dynamically generating concise abstracts and automated metadata summaries.

Roles and Responsibilities

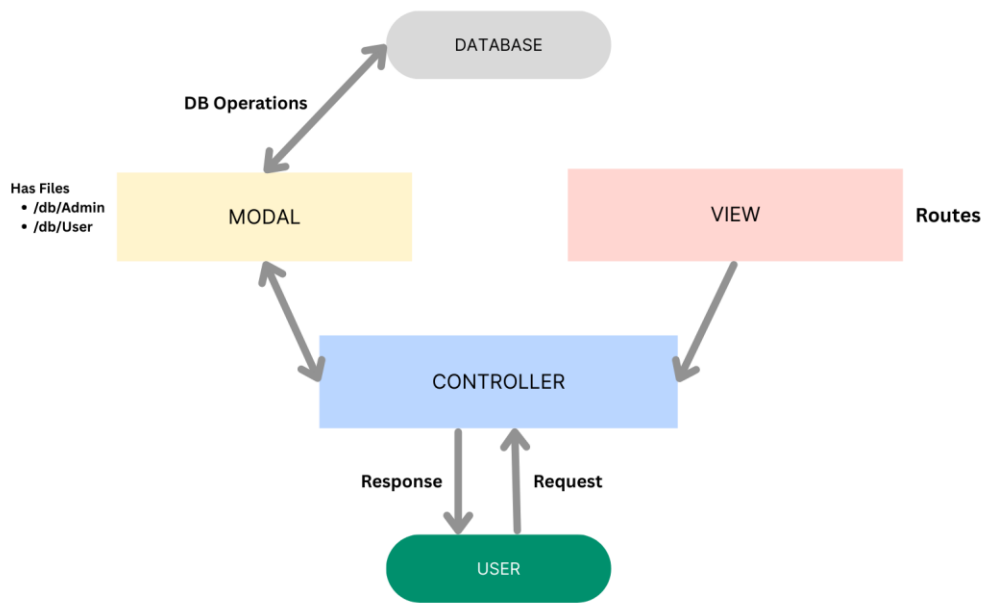
- **Admin:** Complete environment override control. Can manage user database registrations, alter role assignments, prune malicious comments, and inspect full logging entries.
- **Editor:** Focuses on editorial workflows. Reviews pending draft posts, marks records for instant publication, and handles category cleanup scripts.
- **Author:** Restricted account access. Can issue creation statements only for posts bound to their explicit account record.

- **Reader:** Consumer access profile. Reads published materials and posts comments under an ongoing authenticated identity.

User Flow:



MVC Architecture Pattern



The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:

Model Layer (Data Blueprint)

Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets default states, indexing rules, and transformation parameters before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from the execution routes, checks structural request constraints, delegates processing workflows directly to models, and packages the resultant output into standard JSON return arrays.

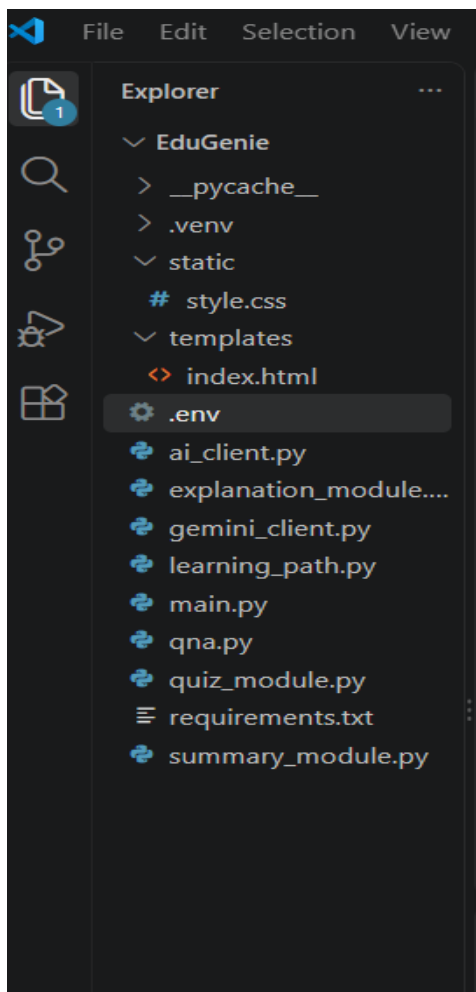
View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API **Routing Layer**. It links network requests on specified paths directly to their associated controller handler logic maps.

Epic2: Project Setup and Configuration

Creating project folder

1. Create a new folder with your <project name>.
2. Inside that folder create one new folders.
3. Name another one as Server.
4. Now open that folder in VS Code.



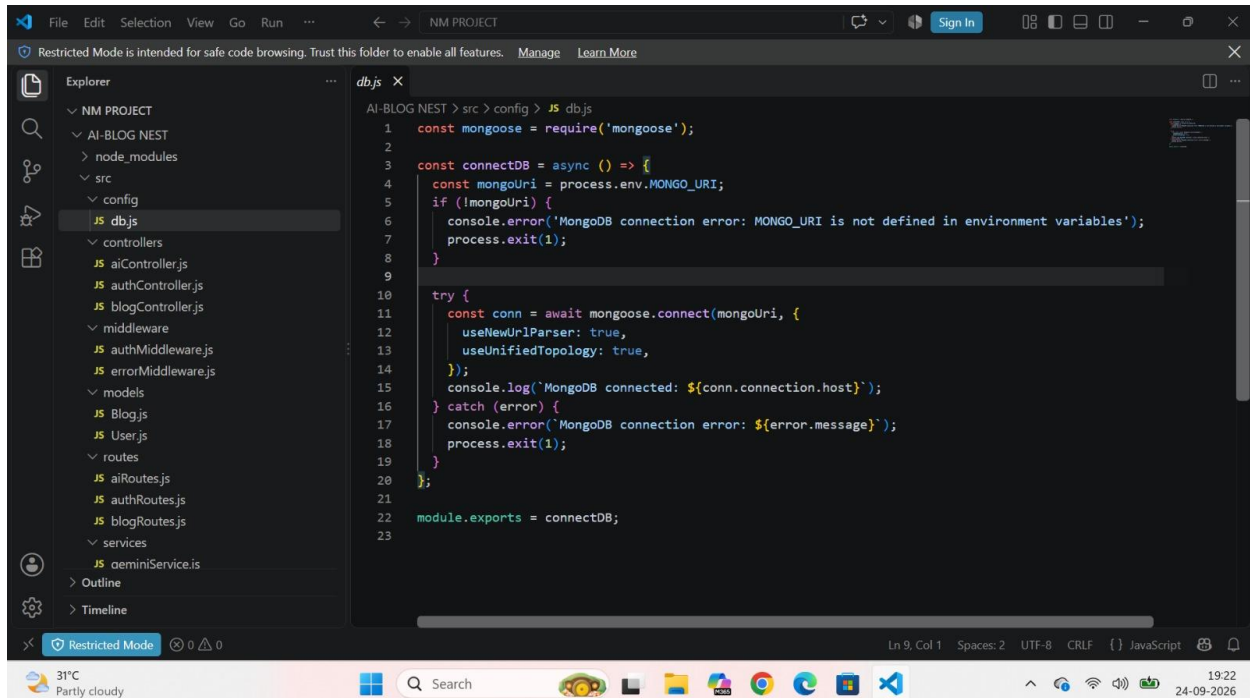
Server setup (npm init)

- Open Server folder in terminal of VScode.
 - npm init -y
- Create files:
 - server.js
- Create folders:
 - models
 - controllers
 - routes

Epic3:BackendDevelopment

Epic4: Database Configuration

DB Connection:-



1. Configure MongoDB:

- Install Mongoose.
In your Node.js project directory, run:
`npm install mongoose`
- Create database connection.
- Create Schemas & Models.

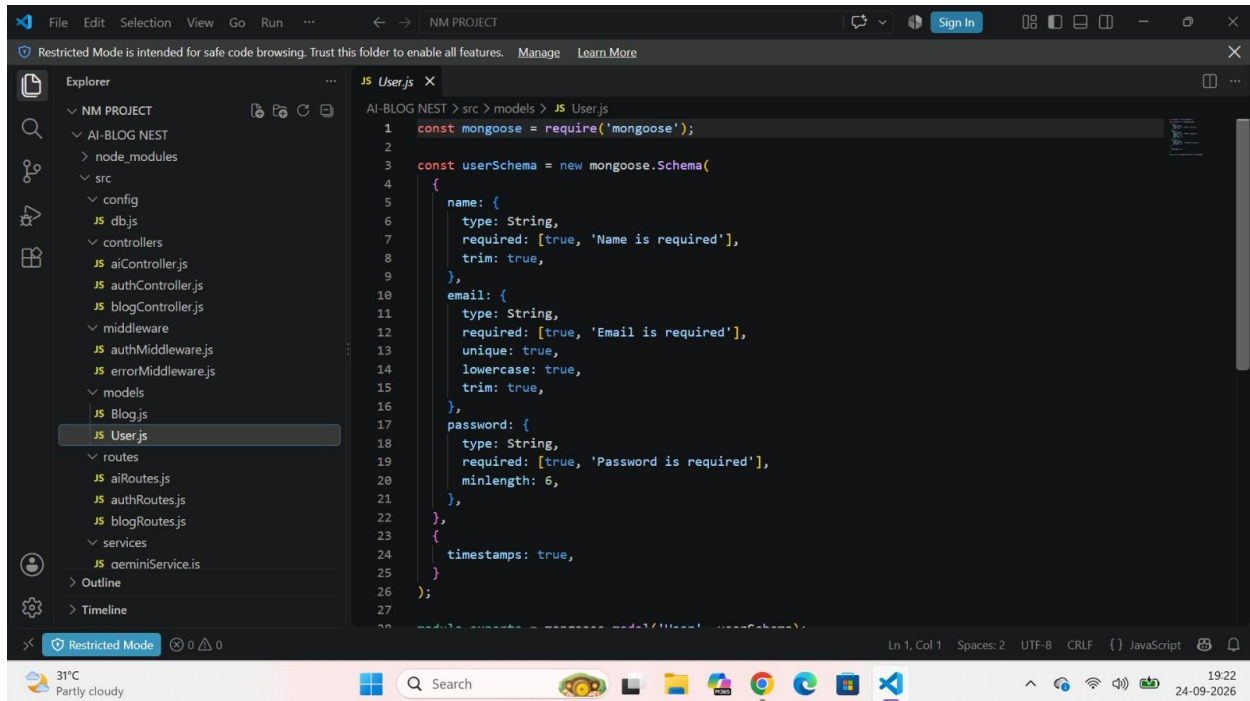
Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions

1. Identity Resource Document Schema (`schemas/userModel.js`)

Enforces field criteria validations on incoming database creations:

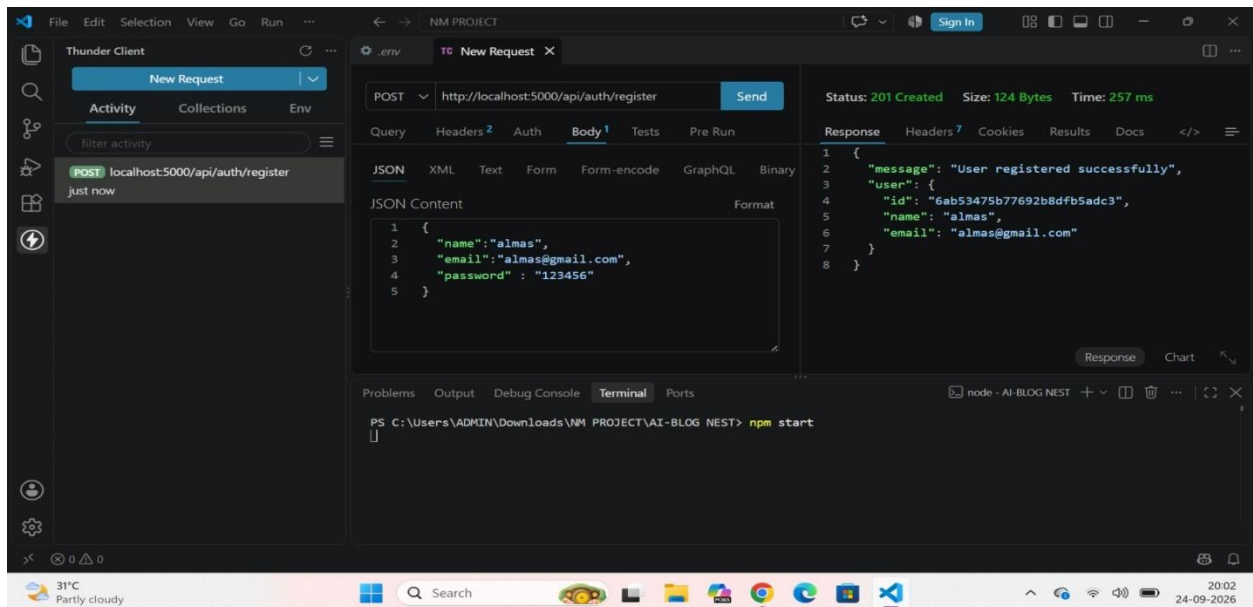


Epic 5: Project Executions:

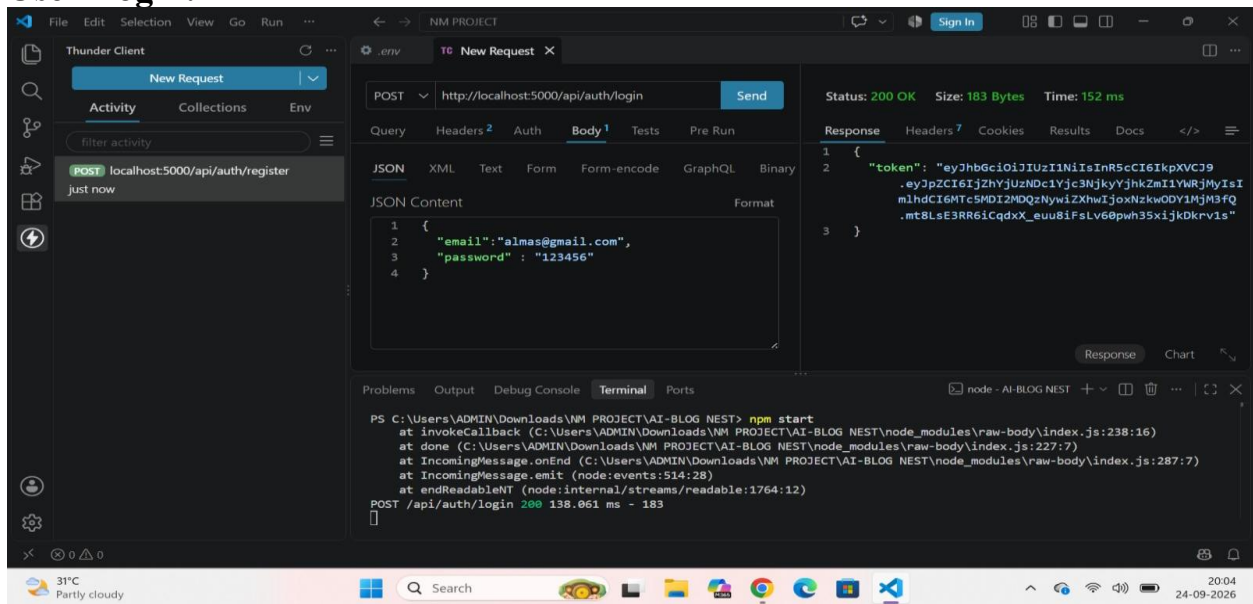
- Install Postman or ThunderClient (VS Code extension).
- Start the backend server:
 “npm start”

API Testing

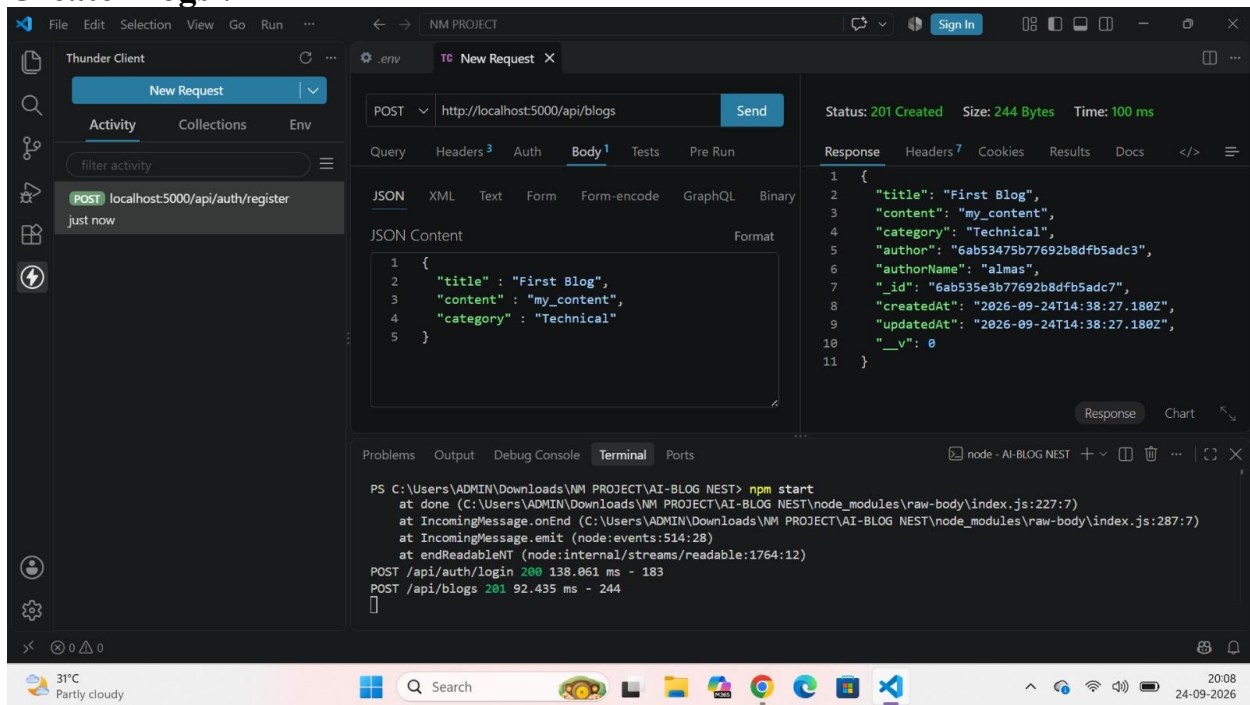
User Registration :



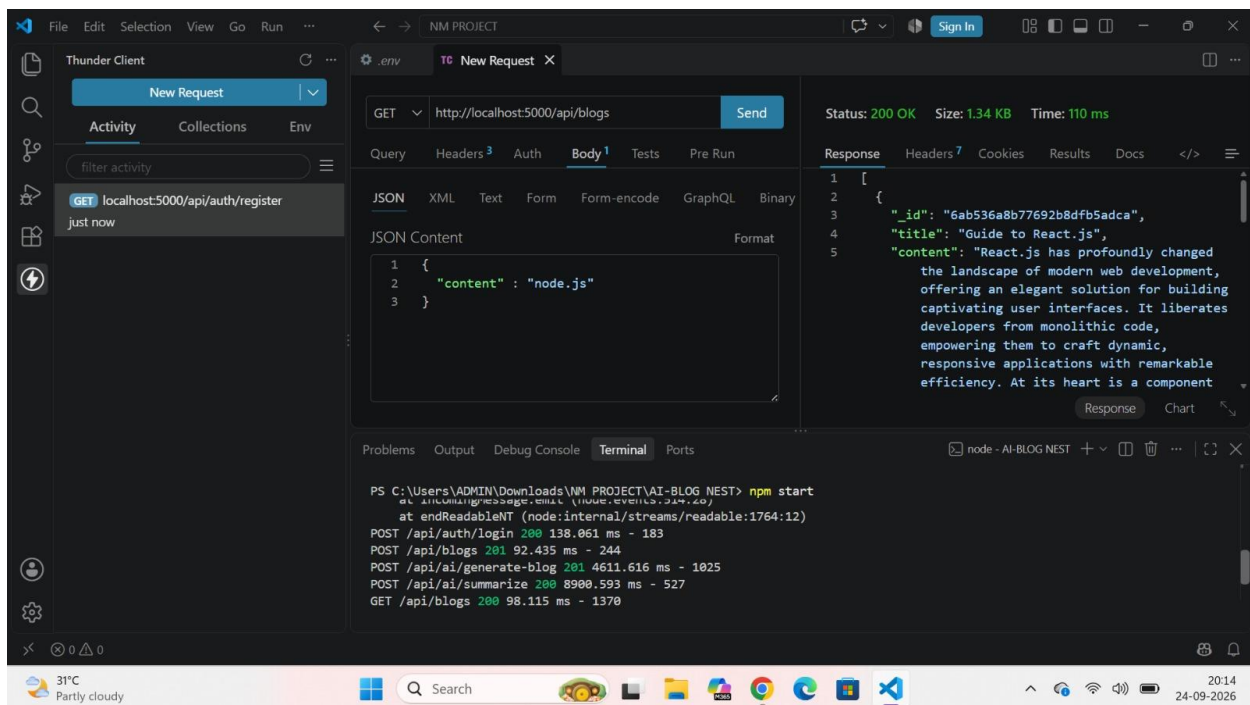
User Login:



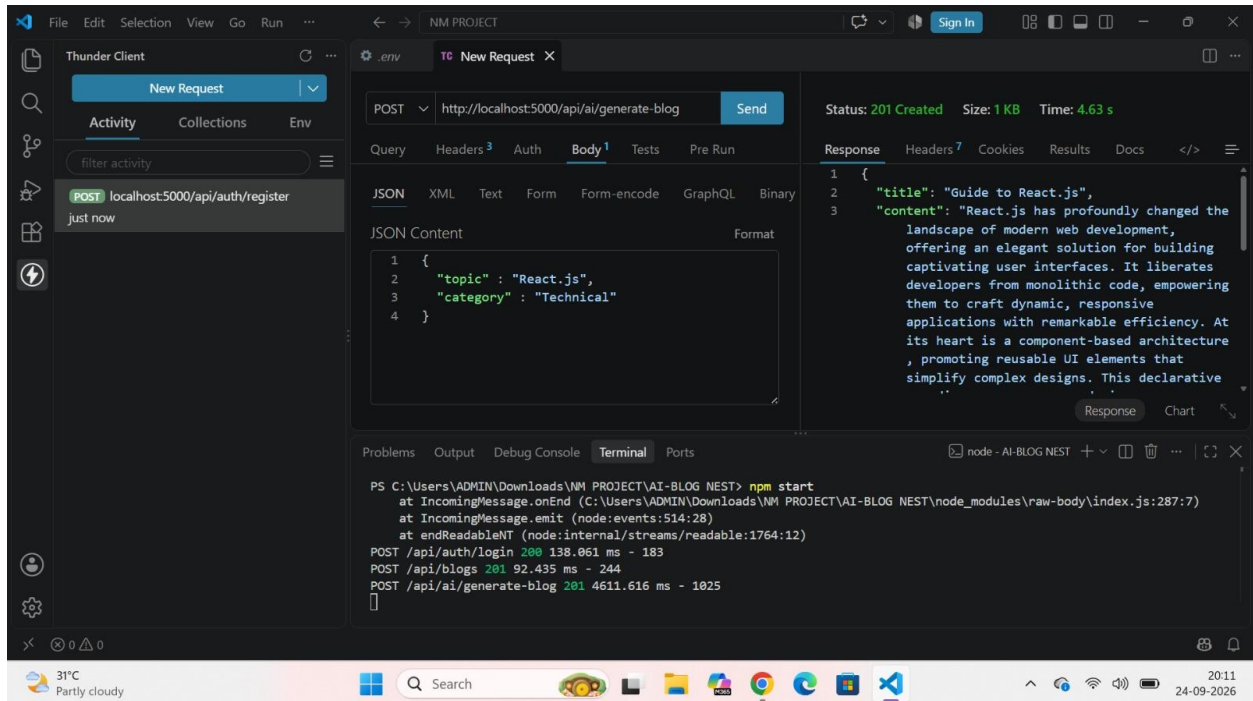
Create Blogs :



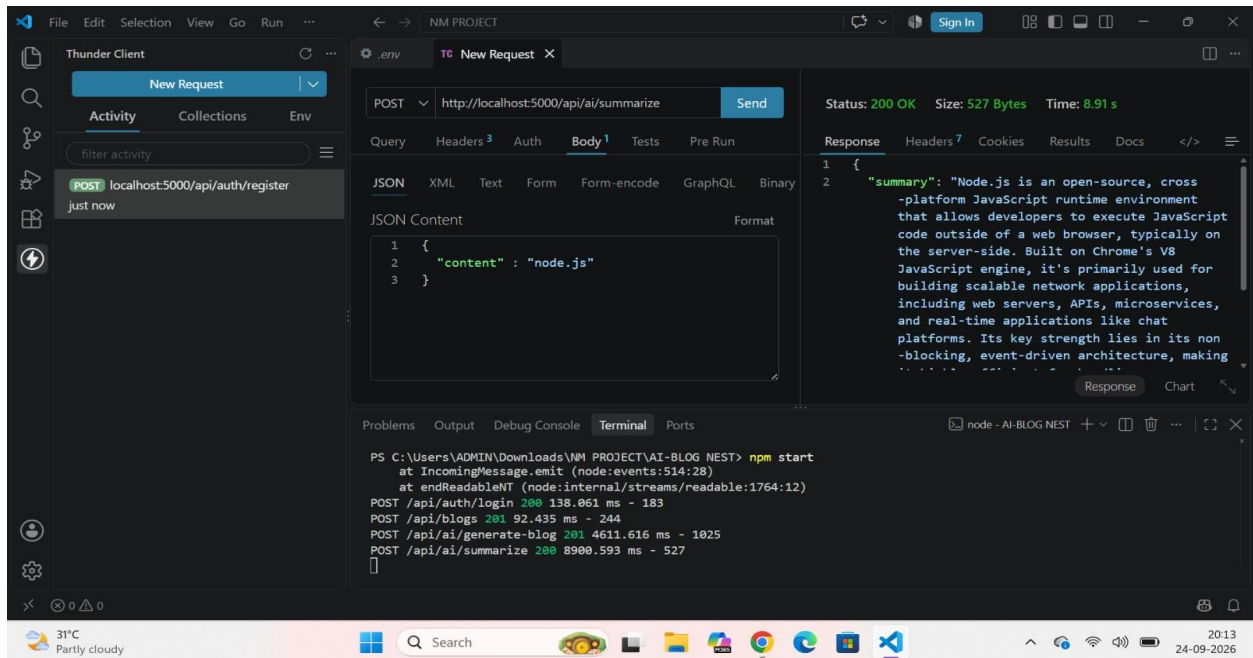
Fetch Blogs :



Generate AI blogs :



Summarize Blogs :



Demo video:

Api testing:

Code drive: